

Exercise 1: CloudFormation Security Review

Task1: Risk Analysis of the Template

The following vulnerabilities were identified during the security review of the provided “template.yaml” file:

Risk 1: Unauthorized Queue

In the “ibanTransactionQueuePolicy”, the usage of wildcard * on “sqs:SendMessage” allows anyone to write to the queue without authentication.

```
12  ibanTransactionQueuePolicy:
13    Type: AWS::SQS::QueuePolicy
14    Properties:
15      PolicyDocument:
16        Version: '2012-10-17'
17        Statement:
18          -
19            Effect: 'Allow'
20            Principal: '*'
21            Action:
22              - 'sqs:SendMessage'
23      Queues:
24        - '|ref|ibanTransactionQueue'
```

This could also lead to Denial of Service attack, as it allows to flood the queue with huge volume of payloads.

Business Impact: Financial Theft and Service Disruption

An attacker could inject fake deposits/transactions or flooding the queue can lead to service paralysis and burning the budget when trying to scale up the flooded data traffic.

Risk 2: Improper Access Control

In the given template, the `AccessControl` is set to “AuthenticatedRead”, which means any AWS user globally. Therefore, anyone who has an AWS account will be able to list and download objects from the bucket.

```
46 bankStatementPdfs:
47   Type: AWS::S3::Bucket
48   BucketName: bankStatements
49   DeletionPolicy: Retain
50   Properties:
51     AccessControl: AuthenticatedRead
52   CorsConfiguration:
53     CorsRules:
```

Business Impact: Confidentiality Loss

All the customer bank statements are exposed to the public. This will impact the GDPR, which leads to massive legal fines and decline in brand value.

Risk3: Over Permissive IAM Role

The usage of “s3:PutObject” and “s3:PutObjectAcl” with `Resource:*` allows this role to overwrite any bucket in the account.

```
88 Statement:
89   - Action:
90     - s3:ListAllMyBuckets
91     - s3:PutObject
92     - s3:PutObjectAcl
93     - s3:GetObject
94     Resource:
95       - arn:aws:s3:::*
96       - arn:aws:s3:::*/*
97     Effect: Allow
```

Business Impact: Data Exfiltration

This role could access backup buckets, and it can lead to no recovery option.

Risk4: Insecure S3 Bucket Configuration

The usage of wildcard * on “AllowedOrigins” and “AllowedHeaders” on the “BankStatementPdfs” bucket allows Cross Origin Resource Sharing (CORS).

This can lead to a browser flow which allows requests from malicious domains and sends data to the attacker.

```
46 bankStatementPdfs:
47   Type: AWS::S3::Bucket
48   BucketName: bankStatements
49   DeletionPolicy: Retain
50   Properties:
51     AccessControl: AuthenticatedRead
52     CorsConfiguration:
53       CorsRules:
54         -
55           AllowedOrigins:
56             - '*'
57           AllowedMethods:
58             - 'GET'
59             - 'POST'
60             - 'PUT'
61           AllowedHeaders:
62             - '*'
```

The same origin policy is disabled for this bucket.

Business Impact: Data Leakage

The user bank statements could be exposed, which leads to heavy legal penalties for failing to meet privacy standards.

Risk5: Hardcoded S3 Bucket Name

The bucket name bankStatements is predictable. This will make it an easier target for the enumeration

```
46     bankStatementPdfs:
47         Type: AWS::S3::Bucket
48         BucketName: bankStatements
49         DeletionPolicy: Retain
```

Business Impact: Potential misconfiguration abuse

If an intruder gets past the first line of defense, they can read everything instantly.

Risk6: Public S3 Bucket Policy

The usage of wildcard * principal on s3:GetObject makes the bucket objects publicly readable.

```
32     websiteStaticAssetPolicy:
33         Type: AWS::S3::BucketPolicy
34         Properties:
35             Bucket: !Ref 'websiteStaticAsset'
36             PolicyDocument:
37                 Statement:
38                     Effect: Allow
39                     Principal: *
40                     Action:
41                         - s3:GetObject
42                     Resource:
43                         - !Sub 'arn:aws:s3:::${websiteStaticAsset}/*'
44                         - !Sub 'arn:aws:s3:::${websiteStaticAsset}'
```

Business Impact: Potential Data Leakage

If this handles sensitive data, it will be publicly readable due to the other existing risks in the template.

Observations:

During the security review, the following observations were made:

- ❖ The infrastructure is missing encryption as it is missing critical properties like “BucketEncryption” and “KmsMasterKeyId”. This could lead to the storage of data in plain-text.
- ❖ Explicit usage of wildcard * principal makes the sensitive data publicly available.

Risk Prioritization

Priority	Risk	Justification
P1 Critical	Unauthorized Queue	Actively exploitable vulnerability that can cause service disruptions and unauthorized transactions
P1 Critical	Improper Access Control	Anyone with an AWS account can access the data. No exploit is required.
P2 High	Over Permissive IAM role	If this role have access to backup files, it could erase the recovery options.
P2 High	Insecure S3 Bucket Configuration	Due to CORS, it allows malicious requests.
P3 Medium	Hardcoded S3 Bucket Name	Predictable bucket names make it an easy target for attackers.
P4 Low	Public S3 Bucket Policy	If this is used on a read-only static page, the severity would be less. However, if this page handles any sensitive information, this could be moved to P2.

Task 2: Incident Response Scenario

If an active attack is detected on the deployed template, following below would be my action plan:

Step1: Initial Response and Containment

- ❖ Immediately block the S3 public access at the account level to override any individual bucket ACLs like AuthenticatedRead
- ❖ Revoke all active sessions for over permissive “iamRole” and any suspicious roles. This helps to halt any lateral movement due to s3:*
- ❖ Isolate the SQS Queue by updating the “ibanTransactionQueuePolicy” to “Deny” for all Principals. This prevents the unauthenticated transactions and flooding.
- ❖ Rotate the current credentials for critical users and roles.

Step2: Investigation and Analysis

- ❖ Use Amazon Athena to query AWS CloudTrail logs for SQS and “s3:GetObject” events. Look for API calls from unknown IPs.
- ❖ Analyse logs for the EC2 instances associated with over permissive “iamRole”. High outbound traffic to unknown IPs could potentially a privilege escalation.

Step3: Recovery

- ❖ Delete the compromised stack and deploy a hardened version using the "Secure-by-Default" template. This includes removing all “Principal: *” references and replacing S3 ACLs with restrictive bucket policies.
- ❖ Audit all the transactions happened during the attack window on the “ibanTransactionQueue” and reverse any fraudulent IBAN entries.

Step4: Prevention and Improvement

- ❖ Harden all the policies by replacing wildcard resources with access only to the specific buckets and queues.
- ❖ Enable AWS Configuration Rules to automatically detect any s3 bucket that are created with improper configurations.
- ❖ Enable AWS GuardDuty and configure real time alerting for unusual activities like huge data export, shadow admin behaviour.

AI Usage:

The security review and incident response strategy for the CloudFormation task were conducted entirely without AI assistance. However, open-source references were done to find the suitable AWS best practices.